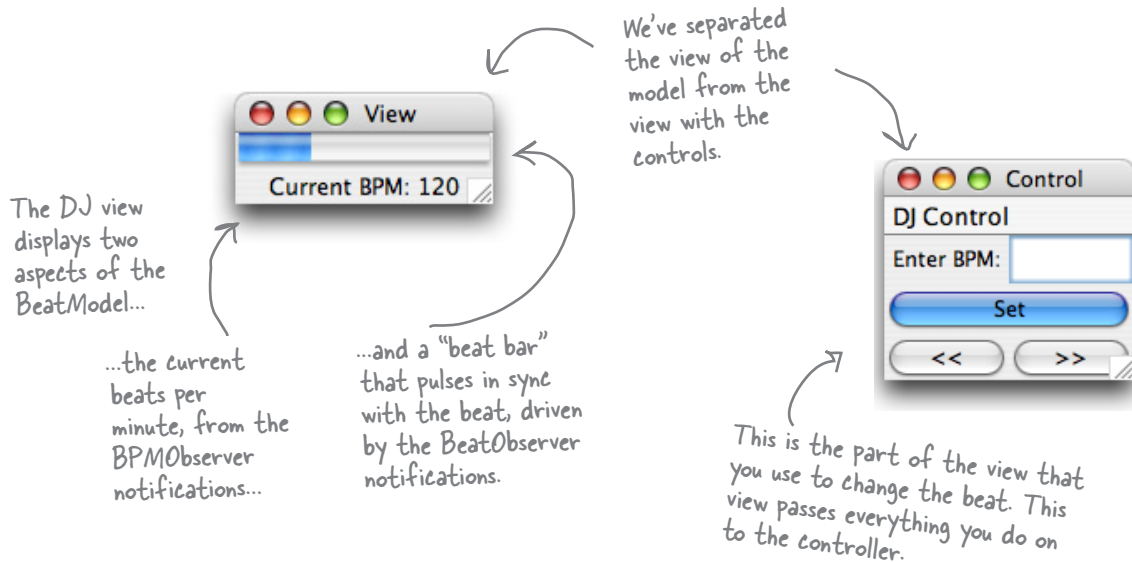


The View

Now the fun starts; we get to hook up a view and visualize the BeatModel!

The first thing to notice about the view is that we've implemented it so that it is displayed in two separate windows. One window contains the current BPM and the pulse; the other contains the interface controls. Why? We wanted to emphasize the difference between the interface that contains the view of the model and the rest of the interface that contains the set of user controls. Let's take a closer look at the two parts of the view:



Our BeatModel makes no assumptions about the view. The model is implemented using the Observer Pattern, so it just notifies any view registered as an observer when its state changes. The view uses the model's API to get access to the state. We've implemented one type of view; can you think of other views that could make use of the notifications and state in the BeatModel?

A light show that is based on the real-time beat.

A textual view that displays a music genre based on the BPM (ambient, downbeat, techno, etc.).

Implementing the View

The two parts of the view—the view of the model, and the view with the user interface controls—are displayed in two windows, but live together in one Java class. We'll first show you just the code that creates the view of the model, which displays the current BPM and the beat bar. Then we'll come back on the next page and show you just the code that creates the user interface controls, which displays the BPM text entry field, and the buttons.



Watch it!

The code on these two pages is just an outline!

What we've done here is split ONE class into TWO, showing you one part of the view on this page, and the other part on the next page. All this code is really in ONE class—DJView.java. It's all listed at the end of the chapter.

DJView is an observer for both real-time beats and BPM changes.

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JFrame viewFrame;
    JPanel viewPanel;
    BeatBar beatBar;
    JLabel bpmOutputLabel;
}
```

The view holds a reference to both the model and the controller. The controller is only used by the control interface, which we'll go over in a sec...

Here, we create a few components for the display.

```
public DJView(ControllerInterface controller, BeatModelInterface model) {
    this.controller = controller;
    this.model = model;
    model.registerObserver((BeatObserver) this);
    model.registerObserver((BPMObserver) this);
}
```

The constructor gets a reference to the controller and the model, and we store references to those in the instance variables.

```
public void createView() {
    // Create all Swing components here
}
```

We also register as a BeatObserver and a BPMObserver of the model.

```
public void updateBPM() {
    int bpm = model.getBPM();
    if (bpm == 0) {
        bpmOutputLabel.setText("offline");
    } else {
        bpmOutputLabel.setText("Current BPM: " + model.getBPM());
    }
}
```

The updateBPM() method is called when a state change occurs in the model. When that happens, we update the display with the current BPM. We can get this value by requesting it directly from the model.

```
public void updateBeat() {
    beatBar.setValue(100);
}
```

Likewise, the updateBeat() method is called when the model starts a new beat. When that happens, we need to pulse our beat bar. We do this by setting it to its maximum value (100) and letting it handle the animation of the pulse.

Implementing the View, continued...

Now, we'll look at the code for the user interface controls part of the view. This view lets you control the model by telling the controller what to do, which in turn, tells the model what to do. Remember, this code is in the same class file as the other view code.

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JLabel bpmLabel;
    JTextField bpmTextField;
    JButton setBPMBButton;
    JButton increaseBPMBButton;
    JButton decreaseBPMBButton;
    JMenuBar menuBar;
    JMenu menu;
    JMenuItem startMenuItem;
    JMenuItem stopMenuItem;

    public void createControls() {
        // Create all Swing components here
    }

    public void enableStopMenuItem() {
        stopMenuItem.setEnabled(true);
    }

    public void disableStopMenuItem() {
        stopMenuItem.setEnabled(false);
    }

    public void enableStartMenuItem() {
        startMenuItem.setEnabled(true);
    }

    public void disableStartMenuItem() {
        startMenuItem.setEnabled(false);
    }

    public void actionPerformed(ActionEvent event) {
        if (event.getSource() == setBPMBButton) {
            int bpm = Integer.parseInt(bpmTextField.getText());
            controller.setBPM(bpm);
        } else if (event.getSource() == increaseBPMBButton) {
            controller.increaseBPM();
        } else if (event.getSource() == decreaseBPMBButton) {
            controller.decreaseBPM();
        }
    }
}
```



This method creates all the controls and places them in the interface. It also takes care of the menu. When the stop or start items are chosen, the corresponding methods are called on the controller.

All these methods allow the start and stop items in the menu to be enabled and disabled. We'll see that the controller uses these to change the interface.

This method is called when a button is clicked.

If the Set button is clicked, then it is passed on to the controller along with the new bpm.

Likewise, if the increase or decrease button is clicked, this information is passed on to the controller.

Now for the Controller

It's time to write the missing piece: the controller. Remember the controller is the strategy that we plug into the view to give it some smarts.

Because we are implementing the Strategy Pattern, we need to start with an interface for any Strategy that might be plugged into the DJ View. We're going to call it `ControllerInterface`.

```
public interface ControllerInterface {  
    void start();  
    void stop();  
    void increaseBPM();  
    void decreaseBPM();  
    void setBPM(int bpm);  
}
```

Here are all the methods the view can call on the controller.

These should look familiar to you after seeing the model's interface. You can stop and start the beat generation and change the BPM. This interface is "richer" than the `BeatModel` interface because you can adjust the BPMs with `increase` and `decrease`.



Design Puzzle

You've seen that the view and controller together make use of the Strategy Pattern. Can you draw a class diagram of the two that represents this pattern?

And here's the implementation of the controller:

```
public class BeatController implements ControllerInterface {
```

```
    BeatModelInterface model;
```

```
    DJView view;
```

```
    public BeatController(BeatModelInterface model) {
```

```
        this.model = model;
```

```
        view = new DJView(this, model);
```

```
        view.createView();
```

```
        view.createControls();
```

```
        view.disableStopMenuItem();
```

```
        view.enableStartMenuItem();
```

```
        model.initialize();
```

```
    }
```

```
    public void start() {
```

```
        model.on();
```

```
        view.disableStartMenuItem();
```

```
        view.enableStopMenuItem();
```

```
    }
```

```
    public void stop() {
```

```
        model.off();
```

```
        view.disableStopMenuItem();
```

```
        view.enableStartMenuItem();
```

```
    }
```

```
    public void increaseBPM() {
```

```
        int bpm = model.getBPM();
```

```
        model.setBPM(bpm + 1);
```

```
    }
```

```
    public void decreaseBPM() {
```

```
        int bpm = model.getBPM();
```

```
        model.setBPM(bpm - 1);
```

```
    }
```

```
    public void setBPM(int bpm) {
```

```
        model.setBPM(bpm);
```

```
    }
```

```
}
```

The controller implements the Controller/Interface.

The controller is the creamy stuff in the middle of the MVC Oreo cookie, so it is the object that gets to hold on to the view and the model and glues it all together.

The controller is passed the model in the constructor and then creates the view.

When you choose Start from the user interface menu, the controller turns the model on and then alters the user interface so that the Start menu item is disabled and the Stop menu item is enabled.

Likewise, when you choose Stop from the menu, the controller turns the model off and alters the user interface so that the Stop menu item is disabled and the Start menu item is enabled.

If the increase button is clicked, the controller gets the current BPM from the model, adds one, and then sets a new BPM.

Same thing here, only we subtract one from the current BPM.

Finally, if the user interface is used to set an arbitrary BPM, the controller instructs the model to set its BPM.

NOTE: the controller is making the intelligent decisions for the view. The view just knows how to turn menu items on and off; it doesn't know the situations in which it should disable them.

Putting it all together...

We've got everything we need: a model, a view, and a controller. Now it's time to put them all together! We're going to see and hear how well they work together.

All we need is a little code to get things started; it won't take much:

```
public class DJTestDrive {
```

```
    public static void main (String[] args) {
```

```
        BeatModelInterface model = new BeatModel();
```

```
        ControllerInterface controller = new BeatController(model);
```

```
    }
```

```
}
```



First create a model...

...then create a controller and pass it the model. Remember, the controller creates the view, so we don't have to do that.

And now for a test run...

Make sure you have the file clip.wav at the top level of the code folder!

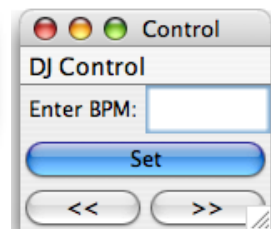
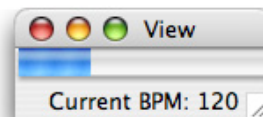
```
File Edit Window Help LetTheBassKick
% java DJTestDrive
%
```

Run this...

...and you'll see this.

Things to try

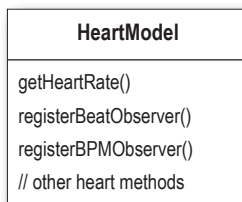
- 1 Start the beat generation with the Start menu item; notice the controller disables the item afterward.
- 2 Use the text entry along with the increase and decrease buttons to change the BPM. Notice how the view display reflects the changes despite the fact that it has no logical link to the controls.
- 3 Notice how the beat bar always keeps up with the beat since it's an observer of the model.
- 4 Put on your favorite song and see if you can match the beat by using the increase and decrease controls.
- 5 Stop the generator. Notice how the controller disables the Stop menu item and enables the Start menu item.



Exploring Strategy

Let's take the Strategy Pattern just a little further to get a better feel for how it is used in MVC. We're going to see another friendly pattern pop up too—a pattern you'll often see hanging around the MVC trio: the Adapter Pattern.

Think for a second about what the DJ View does: it displays a beat rate and a pulse. Does that sound like something else? How about a heartbeat? It just so happens that we have a heart monitor class; here's the class diagram:



We've got a method for getting the current heart rate.

And luckily, its developers knew about the Beat and BPM Observer interfaces!



It certainly would be nice to reuse our current view with the HeartModel, but we need a controller that works with this model. Also, the interface of the HeartModel doesn't match what the view expects because it has a `getHeartRate()` method rather than a `getBPM()`. How would you design a set of classes to allow the view to be reused with the new model? Jot down your class design ideas below.

Adapting the Model

For starters, we're going to need to adapt the HeartModel to a BeatModel. If we don't, the view won't be able to work with the model, because the view only knows how to `getBPM()`, and the equivalent heart model method is `getHeartRate()`. How are we going to do this?

We're going to use the Adapter Pattern, of course! It turns out that this is a common technique when working with MVC: use an adapter to adapt a model to work with existing controllers and views.

Here's the code to adapt a HeartModel to a BeatModel:

```
public class HeartAdapter implements BeatModelInterface {
    HeartModelInterface heart;
```

```
    public HeartAdapter(HeartModelInterface heart) {
        this.heart = heart;
    }
```

```
    public void initialize() {}
```

```
    public void on() {}
```

```
    public void off() {}
```

```
    public int getBPM() {
        return heart.getHeartRate();
    }
```

```
    public void setBPM(int bpm) {}
```

```
    public void registerObserver(BeatObserver o) {
        heart.registerObserver(o);
    }
```

```
    public void removeObserver(BeatObserver o) {
        heart.removeObserver(o);
    }
```

```
    public void registerObserver(BPMObserver o) {
        heart.registerObserver(o);
    }
```

```
    public void removeObserver(BPMObserver o) {
        heart.removeObserver(o);
    }
```

```
}
```

We need to implement the target interface—in this case, `BeatModelInterface`.

Here, we store a reference to the heart model.

We don't know what these would do to a heart, but it sounds scary. So we'll just leave them as "no ops."

When `getBPM()` is called, we'll just translate it to a `getHeartRate()` call on the heart model.

We don't want to do this on a heart! Again, let's leave it as a "no op."

Here are our observer methods. We just delegate them to the wrapped heart model.

Now we're ready for a HeartController

With our HeartAdapter in hand, we should be ready to create a controller and get the view running with the HeartModel. Talk about reuse!

```
public class HeartController implements ControllerInterface {
    HeartModelInterface model;
    DJView view;

    public HeartController(HeartModelInterface model) {
        this.model = model;
        view = new DJView(this, new HeartAdapter(model));
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.disableStartMenuItem();
    }

    public void start() {}

    public void stop() {}

    public void increaseBPM() {}

    public void decreaseBPM() {}

    public void setBPM(int bpm) {}
}
```

The HeartController implements the ControllerInterface, just like the BeatController did.

Like before, the controller creates the view and gets everything glued together.

There is one change: we are passed a HeartModel, not a BeatModel...

...and we need to wrap that model with an adapter before we hand it to the view.

Finally, the HeartController disables the menu items because they aren't needed.

There's not a lot to do here; after all, we can't really control hearts like we can beat machines.

And that's it! Now it's time for some test code...

```
public class HeartTestDrive {

    public static void main (String[] args) {
        HeartModel heartModel = new HeartModel();
        ControllerInterface model = new HeartController(heartModel);
    }
}
```

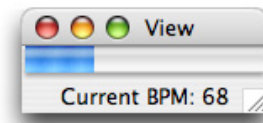
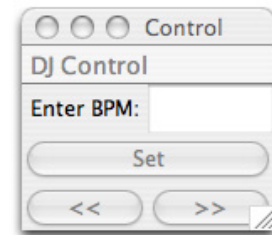
All we need to do is create the controller and pass it a heart monitor.

And now for a test run...

```
File Edit Window Help CheckMyPulse
% java HeartTestDrive
%
```

←
Run this...

...and you'll see this. ↘



↑
Nice healthy
heart rate.

Things to try

- 1 Notice that the display works great with a heart! The beat bar looks just like a pulse. Because the HeartModel also supports BPM and Beat Observers, we can get beat updates just like with the DJ beats.
- 2 As the heartbeat has natural variation, notice the display is updated with the new beats per minute.
- 3 Each time we get a BPM update, the adapter is doing its job of translating getBPM() calls to getHeartRate() calls.
- 4 The Start and Stop menu items are not enabled because the controller disabled them.
- 5 The other buttons still work but have no effect because the controller implements no ops for them. The view could be changed to support the disabling of these items.

there are no Dumb Questions

Q: It seems like you are really hand-waving the fact that the Composite Pattern is really in MVC. Is it really there?

A: Yes, Virginia, there really is a Composite Pattern in MVC. But, actually, this is a very good question. Today GUI packages, like Swing, have become so sophisticated that we hardly notice the internal structure and the use of Composite in the building and update of the display. It's even harder to see when we have web browsers that can take markup language and convert it into a user interface.

Back when MVC was first discovered, creating GUIs required a lot more manual intervention and the pattern was more obviously part of the MVC.

Q: Does the controller ever implement any application logic?

A: No, the controller implements behavior for the view. It is the smarts that translates the actions from the view to actions on the model. The model takes those actions and implements the application logic to decide what to do in response to those actions. The controller might have to do a little work to determine what method calls to make on the model, but that's not considered the "application logic." The application logic is the code that manages and manipulates your data and it lives in your model.

Q: I've always found the word "model" hard to wrap my head around. I now get that it's the guts of the application, but why was such a vague, hard-to-understand word used to describe this aspect of MVC?

A: When MVC was named they needed a word that began with a "M" or otherwise they couldn't have called it MVC.

But seriously, we agree with you. Everyone scratches their head and wonders what a model is. But then everyone comes to the realization that they can't think of a better word either.

Q: You've talked a lot about the state of the model. Does this mean it has the State Pattern in it?

A: No, we mean the general idea of state. But certainly some models do use the State Pattern to manage their internal states.

Q: I've seen descriptions of MVC where the controller is described as a "mediator" between the view and the model. Is the controller implementing the Mediator Pattern?

A: We haven't covered the Mediator Pattern (although you'll find a summary of the pattern in the appendix), so we won't go into too much detail here, but the intent of the mediator is to encapsulate how objects interact and promote loose coupling by keeping two objects from referring to each other explicitly. So, to some degree, the controller can be seen as a mediator, since the view never sets state directly on the model, but rather always goes through the controller. Remember, however, that the view does have a reference to the model to access its state. If the controller were truly a mediator, the view would have to go through the controller to get the state of the model as well.

Q: Does the view always have to ask the model for its state? Couldn't we use the push model and send the model's state with the update notification?

A: Yes, the model could certainly send its state with the notification, and we could do something similar with the BeatModel by sending just the state that the view is interested in. If you remember the Observer Pattern chapter, however, you'll also remember that there are a couple of disadvantages to this. If you don't, go back to Chapter 2 and have a second look. The MVC model has been adapted to a number of similar models—in particular, for the web's browser/server environment—so you'll find a lot of exceptions to the rule out there.

Q: If I have more than one view, do I always need more than one controller?

A: Typically, you need one controller per view at runtime; however, the same controller class can easily manage many views.

Q: The view is not supposed to manipulate the model; however, I noticed in your implementation that the view has full access to the methods that change the model's state. Is this dangerous?

A: You are correct; we gave the view full access to the model's set of methods. We did this to keep things simple, but there may be circumstances where you want to give the view access to only part of your model's API. There's a great design pattern that allows you to adapt an interface to provide only a subset. Can you think of it?



Most of my user interfaces are actually browser-based. Is any of this going to help me?

Yes!

MVC is so useful that it has been adapted to many web frameworks. Of course, the web works differently than your standard application, so there are several different approaches to applying the MVC Pattern to the web.

Web applications have a client side (the browser) and a server side. Given that, we can make different design tradeoffs based on where the model, the view, and the controller reside. In *thin client* approaches, the model, most of the view, and the controller all reside in the server, with the browser providing a way to display the view, and to get input from the browser to the controller. Another approach is the *single page application*, where almost all of the model, view, and controller reside on the client side. Those are the two ends of the spectrum, and you'll find frameworks that vary the extent to which each component—that is the model, the view, and the controller—reside on the client or the server, along with hybrid models where some components are shared across the client and server.

There are many popular web MVC frameworks, like Spring Web MVC, Django, ASP.NET MVC, AngularJS, EmberJS, JavaScriptMVC, Backbone, and no doubt more on the way. For the most part each framework has its own unique way it maps the model, the view, and the controller across the client and the server. Now that you know the MVC Pattern, you will have no problem adapting your knowledge to whatever framework you choose to use.



Tools for your Design Toolbox

You could impress anyone with your design toolbox. Wow, look at all those principles, patterns, and now, compound patterns!

OO Principles

Encapsulate what varies.
 Favor composition over inheritance.
 Program to interfaces, not implementations.
 Strive for loosely coupled designs between objects that interact.
 Classes should be open for extension but closed for modification.
 Depend on abstractions. Do not depend on concrete classes.
 Only talk to your friends.
 Don't call us, we'll call you.
 A class should have only one reason to change.

OO Basics

Abstraction
 Encapsulation
 Polymorphism
 Inheritance

OO Patterns

Proxy - Provide a surrogate or placeholder for another object to control access to it.

Compound Patterns

A Compound Pattern combines two or more patterns into a solution that solves a recurring or general problem.

We have a new category! MVC is a compound pattern.



BULLET POINTS

- The Model View Controller (MVC) Pattern is a compound pattern consisting of the Observer, Strategy, and Composite Patterns.
- The model makes use of the Observer Pattern so that it can keep observers updated yet stay decoupled from them.
- The controller is the Strategy for the view. The view can use different implementations of the controller to get different behavior.
- The view uses the Composite Pattern to implement the user interface, which usually consists of nested components like panels, frames, and buttons.
- These patterns work together to decouple the three players in the MVC model, which keeps designs clear and flexible.
- The Adapter Pattern can be used to adapt a new model to an existing view and controller.
- MVC has been adapted to the web.
- There are many web MVC frameworks with various adaptations of the MVC pattern to fit the client/server application structure.



Exercise Solutions



Sharpen your pencil Solution

The QuackCounter is a Quackable too. When we change Quackable to extend QuackObservable, we have to change every class that implements Quackable, including QuackCounter:

QuackCounter is a Quackable, so now it's a QuackObservable too.

```
public class QuackCounter implements Quackable {
```

```
    Quackable duck;
```

```
    static int numberOfQuacks;
```

```
    public QuackCounter(Quackable duck) {
        this.duck = duck;
    }
```

```
    public void quack() {
        duck.quack();
        numberOfQuacks++;
    }
```

```
    public static int getQuacks() {
        return numberOfQuacks;
    }
```

Here's the duck that QuackCounter is decorating. It's this duck that really needs to handle the observable methods.

All of this code is the same as the previous version of QuackCounter.

```
    public void registerObserver(Observer observer) {
        duck.registerObserver(observer);
    }
```

```
    public void notifyObservers() {
        duck.notifyObservers();
    }
```

```
}
```

Here are the two QuackObservable methods. Notice that we just delegate both calls to the duck that we're decorating.

Sharpen your pencil Solution

What if our Quackologist wants to observe an entire flock? What does that mean anyway? Think about it like this: if we observe a composite, then we're observing everything in the composite. So, when you register with a flock, the flock composite makes sure you get registered with all its children, which may include other flocks.

```
public class Flock implements Quackable {
    List<Quackable> quackers = new ArrayList<Quackable>();
```

Flock is a Quackable, so now
it's a QuackObservable too.

```
    public void add(Quackable duck) {
        ducks.add(duck);
    }
```

Here are the Quackables
that are in the Flock.

```
    public void quack() {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable duck = iterator.next();
            duck.quack();
        }
    }
```

When you register as an Observer
with the Flock, you actually
get registered with everything
that's IN the flock, which is
every Quackable, whether it's a
duck or another Flock.

```
    public void registerObserver(Observer observer) {
        Iterator<Quackable> iterator = ducks.iterator();
        while (iterator.hasNext()) {
            Quackable duck = iterator.next();
            duck.registerObserver(observer);
        }
    }
```

We iterate through all the
Quackables in the Flock
and delegate the call to
each Quackable. If the
Quackable is another Flock,
it will do the same.

```
    public void notifyObservers() { }
```

Each Quackable does its own notification, so
Flock doesn't have to worry about it. This
happens when Flock delegates quack() to each
Quackable in the Flock.

Sharpen your pencil Solution



We're still directly instantiating Geese by relying on concrete classes. Can you write an Abstract Factory for Geese? How should it handle creating "goose ducks"?

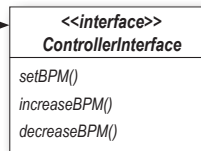
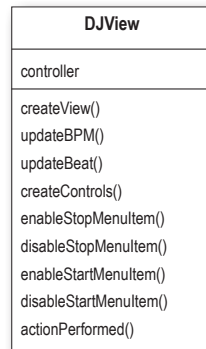
You could add a createGooseDuck() method to the existing Duck Factories. Or, you could create a completely separate Factory for creating families of Geese.



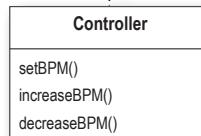
Design Puzzle Solution

You've seen that the view and controller together make use of the Strategy Pattern. Can you draw a class diagram of the two that represents this pattern?

The view delegates behavior to the controller. The behavior it delegates is how to control the model based on user input.



The ControllerInterface is the interface that all concrete controllers implement. This is the strategy interface.



We can plug in different controllers to provide different behaviors for the view.



Here's the complete implementation of the DJView. It shows all the MIDI code to generate the sound, and all the Swing components to create the view. You can also download this code at <https://www.wickedlysmart.com>. Have fun!

```
package headfirst.designpatterns.combined.djview;

public class DJTestDrive {

    public static void main (String[] args) {
        BeatModelInterface model = new BeatModel();
        ControllerInterface controller = new BeatController(model);
    }
}
```

The Beat Model

```
package headfirst.designpatterns.combined.djview;

public interface BeatModelInterface {
    void initialize();

    void on();

    void off();

    void setBPM(int bpm);

    int getBPM();

    void registerObserver(BeatObserver o);

    void removeObserver(BeatObserver o);

    void registerObserver(BPMObserver o);

    void removeObserver(BPMObserver o);
}
```

```
package headfirst.designpatterns.combined.djview;

import java.util.*;
import javax.sound.sampled.AudioSystem;
import javax.sound.sampled.Clip;
import java.io.*;
import javax.sound.sampled.Line;

public class BeatModel implements BeatModelInterface, Runnable {
    List<BeatObserver> beatObservers = new ArrayList<BeatObserver>();
    List<BPMObserver> bpmObservers = new ArrayList<BPMObserver>();
    int bpm = 90;
    Thread thread;
    boolean stop = false;
    Clip clip;

    public void initialize() {
        try {
            File resource = new File("clap.wav");
            clip = (Clip) AudioSystem.getLine(new Line.Info(Clip.class));
            clip.open(AudioSystem.getAudioInputStream(resource));
        }
        catch (Exception ex) {
            System.out.println("Error: Can't load clip");
            System.out.println(ex);
        }
    }

    public void on() {
        bpm = 90;
        notifyBPMObservers();
        thread = new Thread(this);
        stop = false;
        thread.start();
    }

    public void off() {
        stopBeat();
        stop = true;
    }
}
```



Ready Bake Code

```

public void run() {
    while (!stop) {
        playBeat();
        notifyBeatObservers();
        try {
            Thread.sleep(60000/getBPM());
        } catch (Exception e) {}
    }
}

public void setBPM(int bpm) {
    this.bpm = bpm;
    notifyBPMObservers();
}

public int getBPM() {
    return bpm;
}

public void registerObserver(BeatObserver o) {
    beatObservers.add(o);
}

public void notifyBeatObservers() {
    for (int i = 0; i < beatObservers.size(); i++) {
        BeatObserver observer = (BeatObserver)beatObservers.get(i);
        observer.updateBeat();
    }
}

public void registerObserver(BPMObserver o) {
    bpmObservers.add(o);
}

public void notifyBPMObservers() {
    for (int i = 0; i < bpmObservers.size(); i++) {
        BPMObserver observer = (BPMObserver)bpmObservers.get(i);
        observer.updateBPM();
    }
}

```

```
public void removeObserver(BeatObserver o) {
    int i = beatObservers.indexOf(o);
    if (i >= 0) {
        beatObservers.remove(i);
    }
}

public void removeObserver(BPMObserver o) {
    int i = bpmObservers.indexOf(o);
    if (i >= 0) {
        bpmObservers.remove(i);
    }
}

public void playBeat() {
    clip.setFramePosition(0);
    clip.start();
}

public void stopBeat() {
    clip.setFramePosition(0);
    clip.stop();
}

}
```

The View



```
package headfirst.designpatterns.combined.djview;

public interface BeatObserver {
    void updateBeat();
}

package headfirst.designpatterns.combined.djview;

public interface BPMObserver {
    void updateBPM();
}

package headfirst.designpatterns.combined.djview;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JFrame viewFrame;
    JPanel viewPanel;
    BeatBar beatBar;
    JLabel bpmOutputLabel;
    JFrame controlFrame;
    JPanel controlPanel;
    JLabel bpmLabel;
    JTextField bpmTextField;
    JButton setBPMBButton;
    JButton increaseBPMBButton;
    JButton decreaseBPMBButton;
    JMenuBar menuBar;
    JMenu menu;
    JMenuItem startMenuItem;
    JMenuItem stopMenuItem;

    public DJView(ControllerInterface controller, BeatModelInterface model) {
        this.controller = controller;
        this.model = model;
        model.registerObserver((BeatObserver) this);
        model.registerObserver((BPMObserver) this);
    }
}
```

```

public void createView() {
    // Create all Swing components here
    viewPanel = new JPanel(new GridLayout(1, 2));
    viewFrame = new JFrame("View");
    viewFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    viewFrame.setSize(new Dimension(100, 80));
    bpmOutputLabel = new JLabel("offline", SwingConstants.CENTER);
    beatBar = new BeatBar();
    beatBar.setValue(0);
    JPanel bpmPanel = new JPanel(new GridLayout(2, 1));
    bpmPanel.add(beatBar);
    bpmPanel.add(bpmOutputLabel);
    viewPanel.add(bpmPanel);
    viewFrame.getContentPane().add(viewPanel, BorderLayout.CENTER);
    viewFrame.pack();
    viewFrame.setVisible(true);
}

public void createControls() {
    // Create all Swing components here
    JFrame.setDefaultLookAndFeelDecorated(true);
    controlFrame = new JFrame("Control");
    controlFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    controlFrame.setSize(new Dimension(100, 80));

    controlPanel = new JPanel(new GridLayout(1, 2));

    menuBar = new JMenuBar();
    menu = new JMenu("DJ Control");
    startMenuItem = new JMenuItem("Start");
    menu.add(startMenuItem);
    startMenuItem.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            controller.start();
        }
    });
    stopMenuItem = new JMenuItem("Stop");
    menu.add(stopMenuItem);
    stopMenuItem.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            controller.stop();
        }
    });
    JMenuItem exit = new JMenuItem("Quit");
    exit.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            System.exit(0);
        }
    });
}

```



Ready Bake Cope

```

menu.add(exit);
menuBar.add(menu);
controlFrame.setJMenuBar(menuBar);

bpmTextField = new JTextField(2);
bpmLabel = new JLabel("Enter BPM:", SwingConstants.RIGHT);
setBPMBButton = new JButton("Set");
setBPMBButton.setSize(new Dimension(10,40));
increaseBPMBButton = new JButton(">>");
decreaseBPMBButton = new JButton("<<");
setBPMBButton.addActionListener(this);
increaseBPMBButton.addActionListener(this);
decreaseBPMBButton.addActionListener(this);

JPanel buttonPanel = new JPanel(new GridLayout(1, 2));
buttonPanel.add(decreaseBPMBButton);
buttonPanel.add(increaseBPMBButton);

JPanel enterPanel = new JPanel(new GridLayout(1, 2));
enterPanel.add(bpmLabel);
enterPanel.add(bpmTextField);
JPanel insideControlPanel = new JPanel(new GridLayout(3, 1));
insideControlPanel.add(enterPanel);
insideControlPanel.add(setBPMBButton);
insideControlPanel.add(buttonPanel);
controlPanel.add(insideControlPanel);

bpmLabel.setBorder(BorderFactory.createEmptyBorder(5,5,5,5));
bpmOutputLabel.setBorder(BorderFactory.createEmptyBorder(5,5,5,5));

controlFrame.getRootPane().setDefaultButton(setBPMBButton);
controlFrame.getContentPane().add(controlPanel, BorderLayout.CENTER);

controlFrame.pack();
controlFrame.setVisible(true);
}

public void enableStopMenuItem() {
    stopMenuItem.setEnabled(true);
}

public void disableStopMenuItem() {
    stopMenuItem.setEnabled(false);
}

```

```
public void enableStartMenuItem() {
    startMenuItem.setEnabled(true);
}

public void disableStartMenuItem() {
    startMenuItem.setEnabled(false);
}

public void actionPerformed(ActionEvent event) {
    if (event.getSource() == setBPMBButton) {
        int bpm = 90;
        String bpmText = bpmTextField.getText();
        if (bpmText == null || bpmText.contentEquals("")) {
            bpm = 90;
        } else {
            bpm = Integer.parseInt(bpmTextField.getText());
        }
        controller.setBPM(bpm);
    } else if (event.getSource() == increaseBPMBButton) {
        controller.increaseBPM();
    } else if (event.getSource() == decreaseBPMBButton) {
        controller.decreaseBPM();
    }
}

public void updateBPM() {
    int bpm = model.getBPM();
    if (bpm == 0) {
        bpmOutputLabel.setText("offline");
    } else {
        bpmOutputLabel.setText("Current BPM: " + model.getBPM());
    }
}

public void updateBeat() {
    beatBar.setValue(100);
}
}
```

The Controller

```
package headfirst.designpatterns.combined.djview;

public interface ControllerInterface {
    void start();
    void stop();
    void increaseBPM();
    void decreaseBPM();
    void setBPM(int bpm);
}
```



```

package headfirst.designpatterns.combined.djview;

public class BeatController implements ControllerInterface {
    BeatModelInterface model;
    DJView view;

    public BeatController(BeatModelInterface model) {
        this.model = model;
        view = new DJView(this, model);
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
        model.initialize();
    }

    public void start() {
        model.on();
        view.disableStartMenuItem();
        view.enableStopMenuItem();
    }

    public void stop() {
        model.off();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
    }

    public void increaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm + 1);
    }

    public void decreaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm - 1);
    }

    public void setBPM(int bpm) {
        model.setBPM(bpm);
    }
}

```

The Heart Model

```
package headfirst.designpatterns.combined.djview;

public class HeartTestDrive {

    public static void main (String[] args) {
        HeartModel heartModel = new HeartModel();
        ControllerInterface model = new HeartController(heartModel);
    }
}

package headfirst.designpatterns.combined.djview;

public interface HeartModelInterface {
    int getHeartRate();
    void registerObserver(BeatObserver o);
    void removeObserver(BeatObserver o);
    void registerObserver(BPMObserver o);
    void removeObserver(BPMObserver o);
}

package headfirst.designpatterns.combined.djview;

import java.util.*;

public class HeartModel implements HeartModelInterface, Runnable {
    List<BeatObserver> beatObservers = new ArrayList<BeatObserver>();
    List<BPMObserver> bpmObservers = new ArrayList<BPMObserver>();
    int time = 1000;
    int bpm = 90;
    Random random = new Random(System.currentTimeMillis());
    Thread thread;

    public HeartModel() {
        thread = new Thread(this);
        thread.start();
    }

    public void run() {
        int lastrate = -1;

        for(;;) {
            int change = random.nextInt(10);
            if (random.nextInt(2) == 0) {
                change = 0 - change;
            }
            int rate = 60000/(time + change);
```



Ready Bake
Code

```

        if (rate < 120 && rate > 50) {
            time += change;
            notifyBeatObservers();
            if (rate != lastrate) {
                lastrate = rate;
                notifyBPMObservers();
            }
        }
        try {
            Thread.sleep(time);
        } catch (Exception e) {}
    }
}

public int getHeartRate() {
    return 60000/time;
}

public void registerObserver(BeatObserver o) {
    beatObservers.add(o);
}

public void removeObserver(BeatObserver o) {
    int i = beatObservers.indexOf(o);
    if (i >= 0) {
        beatObservers.remove(i);
    }
}

public void notifyBeatObservers() {
    for(int i = 0; i < beatObservers.size(); i++) {
        BeatObserver observer = (BeatObserver)beatObservers.get(i);
        observer.updateBeat();
    }
}

public void registerObserver(BPMObserver o) {
    bpmObservers.add(o);
}

public void removeObserver(BPMObserver o) {
    int i = bpmObservers.indexOf(o);
    if (i >= 0) {
        bpmObservers.remove(i);
    }
}

public void notifyBPMObservers() {
    for(int i = 0; i < bpmObservers.size(); i++) {
        BPMObserver observer = (BPMObserver)bpmObservers.get(i);
        observer.updateBPM();
    }
}
}

```

The Heart Adapter

```
package headfirst.designpatterns.combined.djview;

public class HeartAdapter implements BeatModelInterface {
    HeartModelInterface heart;

    public HeartAdapter(HeartModelInterface heart) {
        this.heart = heart;
    }

    public void initialize() {}

    public void on() {}

    public void off() {}

    public int getBPM() {
        return heart.getHeartRate();
    }

    public void setBPM(int bpm) {}

    public void registerObserver(BeatObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BeatObserver o) {
        heart.removeObserver(o);
    }

    public void registerObserver(BPMObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BPMObserver o) {
        heart.removeObserver(o);
    }
}
```

The Controller



```
package headfirst.designpatterns.combined.djview;

public class HeartController implements ControllerInterface {
    HeartModelInterface model;
    DJView view;

    public HeartController(HeartModelInterface model) {
        this.model = model;
        view = new DJView(this, new HeartAdapter(model));
        view.createView();
        view.createControls();
        view.disableStopMenuItem();
        view.disableStartMenuItem();
    }

    public void start() {}

    public void stop() {}

    public void increaseBPM() {}

    public void decreaseBPM() {}

    public void setBPM(int bpm) {}
}
```

